Table of Contents

What Is Serialized, What Is Pass-Through, and What Is Actually Rendered (2026-10-30)
TL;DR
1. Method
1.1 Why three axes
1.2 The corpus
1.3 The three detectors
1.4 Code evidence
2. The package-level map
2.1 The full part matrix
3. Three architectures
3.1 Clone-and-override — Folio
3.2 Byte-splice of untouched blocks — GenOffice and eigenpal
3.3 Full regeneration — office-open, Docen, canvas-editor, docs.plus, Fidus Writer, WordCraft, Plate.js
3.4 The opaque-blob fallback — SuperDoc
4. Per editor: serialized, pass-through, rendered
4.1 SuperDoc — keeps everything, renders the least
4.2 Folio — copies the most, rewrites the least, and still gets one thing fatally wrong
4.3 GenOffice — byte-exact splice, and two visible losses
4.4 eigenpal — canonical tree, projection-only rendering
4.5 office-open and Docen — generators, with no renderer
4.6 WordCraft — regenerates the text, passes the payload through, and redraws the payload itself
4.7 Fidus Writer — the output is the template's
4.8 canvas-editor + plugin — a bespoke parser and a third-party writer
4.9 docs.plus — the document becomes HTML and comes back
4.10 Plate.js — an HTML-shaped editor with a byte-exactness gate
4.11 The second axis that matters: what the editor says
5. The cross-cutting finding: kept but not rendered
6. The rendered comparison
7. What this means
Reproducing this

What Is Serialized, What Is Pass-Through, and What Is Actually Rendered (2026-10-30)

A companion to *Open-Source DOCX Editors, Re-verified Against Code and Measured by Round-Trip (2026-10-29)*. That report asked how faithfully each editor preserved a document. This one asks a different question, one the fidelity score cannot answer: **where does each part of the file come from, and what does the user actually see?**

TL;DR

- **Three axes, not one.** "Kept" is three different things. A part can be **serialized** (rebuilt from a model), **pass-through** (copied byte-for-byte without being understood), or **rendered** (turned into pixels on screen). An editor can score 100% on the first, copy 34 of 35 parts verbatim on the second, and still render a placeholder where a SmartArt diagram should be.
- **SuperDoc renders the least of what it keeps.** It is the only editor with a 100% score, and it keeps all 35 parts and all ten payload parts. But its drawing taxonomy is explicitly *fail-closed* for SmartArt and for OLE objects (drawing-taxonomy.ts:327-333, :320-326): it preserves them in the file and puts a labelled placeholder on screen, with a diagnostic code (render.embedded-object-not-supported).
- **Folio writes the least of what it copies.** 34 of its 35 output parts are byte-identical to the input — and they are copied as **raw compressed bytes**, not decompressed and re-zipped (docx-core/src/zip/archive.ts:388-405: same CRC, same compressed size). Only word/document.xml is rebuilt.
- **The rendered result is the axis that separates them.** Rendering every output and diffing against the corpus's own rendering: SuperDoc, Folio and eigenpal reproduce it **line for line** (81 lines, 265 words, 6 pages, zero missing, zero extra). office-open loses 12 rendered lines while scoring **100% text retention** on the XML level, because its chart relationship dangles and the chart never draws. Fidus Writer, canvas-editor and Plate.js collapse to 2 pages; docs.plus reflows the document into 3 pages with 61 lines that were never in the corpus.
- **Plate.js (16,680★, the second-most-starred project in the trial) is the new extreme on all three axes at once.** It is the only editor with **zero** byte-identical parts, the only one that drops 24 of 35 parts *and adds nothing*, and the only one that renders in **2 pages** while keeping 91.7% of the text. Its pipeline is **DOCX → HTML → model → HTML → DOCX**, and HTML cannot carry a header, a footer, a note, a comment, a chart, a SmartArt diagram, an OLE object, an embedded workbook or an image — so all of those leave the package on the way through.
- **Plate.js is also the only editor that says what it did.** Its import emits **38 typed diagnostics** naming each lost construct, and its lossPolicy defaults to 'reject', so by default it **refuses** a document it cannot represent losslessly instead of writing a degraded copy. It also holds the only implementation of *provable* byte-exact export in this trial — a **closed passive vocabulary** in internal/sourceEligibility.ts:453-582 that the corpus **fails** (a chart relationship is not passive), so Plate declines the byte-exact path and regenerates, saying why.
- **Four of the twelve outputs cannot be opened by Microsoft Word at all.** Folio, office-open, Docen and Fidus Writer are refused by Word 16.115 — and two of those four (Folio and office-open) are *schema-clean, well-formed, and free of dangling references*, so no static validator reports anything at all about them. Plate.js's output **does** open in Word.

1. Method

1.1 Why three axes

A single "did it survive?" check collapses three different engineering decisions:

1. **Serialization** — the editor held the construct in a model and wrote it back out. It understood it, at least partially, and could have changed it.
2. **Pass-through** — the editor never looked at the bytes. It copied them because it had no reason to touch them. This is the strongest possible fidelity, and the cheapest: the cost of understanding a part is only worth paying if you intend to change it.
3. **Rendering** — the editor drew something. This is what the user sees, and it is orthogonal to the other two: a construct can be serialized faithfully *and* rendered as a grey box, or rendered perfectly *and* not survive the round trip.

The first report measured the first axis with 97 checks. This report measures all three.

1.2 The corpus

The same 35-part adversarial corpus (fixture/complex.docx), now corrected so that Microsoft Word 16.115 opens it. It contains a chart, four SmartArt parts, an OLE object with a real embedded .docx, an embedded workbook, an image, two tables (nested, merged, header rows, vertical merge), tracked insertions/deletions/runs/paragraph-marks/moves, content controls (including data-bound, date-picker and nested), complex and simple fields, bookmarks, footnotes, endnotes, comments, OMML math, mc:AlternateContent with both branches, an w:altChunk, w:permStart/w:permEnd, w:proofErr, w14:* attributes and elements, w16du:dateUtc, w16cid:durableId, bidi and RTL runs, three sections including a landscape one with page borders.

1.3 The three detectors

Tool	What it does
build/part_map.py	Compares every part of every output against the input and classifies it: **P** byte-identical pass-through, **w** rewritten (bytes differ, XML tree identical), **C** changed (tree differs), **D** dropped, **A** added. XML comparison is prefix-independent, so a writer that renames namespaces is not mistaken for a writer that changes content.
build/render_compare.py	Renders the corpus and every output to PDF with LibreOffice, extracts the laid-out text with pdftotext -layout, and diffs it. This measures the third axis on the round-trip *result*.
build/doc_diff.py	Walks the corpus's and the output's word/document.xml trees in parallel and reports the elements and attributes that appeared, vanished or changed, with their position.

The rendered comparison uses LibreOffice as the renderer, not the editors. The claim it supports is therefore precise: *what survives into a rendered rendering of the editor's output* — which is the practical consequence of what the editor did with the file. The code-level evidence in §4 covers what each editor itself draws.

1.4 Code evidence

For every editor, the save path and the render path were read in its own source, at the pinned commit from the first report. Every claim below carries a file:line. Where a behaviour could not be established from source, it is marked as a gap rather than inferred.

One structural note that matters for reading §4: **SuperDoc's DOCX engine is proprietary.** The repository depends on @superdoc/docx-engine 0.27.0-next.15, whose licence forbids deobfuscation, so its internal serialization decisions are not auditable from source. Its *outer* contract is auditable, and that is what §4.1 reports.

---

2. The package-level map

P byte-identical pass-through · w rewritten, same XML tree · C changed · D dropped · A added · · not present

Editor	Pass-through	Rewritten	Changed	Dropped	Added	Parts out
Folio	**34**	0	1	0	0	35
SuperDoc	**27**	1	7	0	0	35
GenOffice	20	13	2	0	0	35
WordCraft	16	0	16	3	4	36
office-open	9	10	16	0	0	35
Docen	9	9	17	0	4	39
eigenpal	5	16	14	0	0	35
LibreOffice (baseline)	2	3	20	10	18	43
Fidus Writer	0	0	11	24	3	14
canvas-editor + plugin	0	0	14	21	6	20
docs.plus	0	0	11	24	2	13
Plate.js	**0**	**0**	**11**	**24**	**0**	**11**

Read this table with the first report's scores and it stops being a ranking. **Folio copies more of the input byte-for-byte than any editor, and still scores 98.5%, because the one part it rebuilds — word/document.xml — is where almost all of the interesting markup lives.** Pass-through is a measure of what the editor *ignored*, not of what it preserved.

Plate.js is the new extreme on this axis, and it is worth reading alongside Fidus, canvas-editor and docs.plus. Those four all show 0 pass-through, and all four regenerate the package from a model — but Plate is the only one of them that **adds nothing at all** (0 added, 11 parts out, versus Fidus's 14, canvas-editor's 20 and docs.plus's 13). Its output is the smallest regenerated package in the trial, and it is smaller than the input by a third. The reason is in its architecture: the DOCX pipeline is **DOCX → HTML → editor model → static HTML → DOCX**, and HTML cannot carry a header, a footer, a footnote, an endnote, a comment, a glossary part, a customXml part, an altChunk part, a chart, a SmartArt diagram, an OLE object, an embedded workbook or an image reference — so all twenty-four of those parts leave the package on the way through.

2.1 The full part matrix

The matrix is printed as four tables — two editor halves x two part groups — because thirteen columns do not survive a landscape page: the renderer wraps every header onto four lines and the part names wrap as well. Two conventions keep the cells to one glyph each. **Part names are printed without the word/ prefix** (so word/document.xml is document.xml), with the sub-directories compacted (embeddings/ to embed/, diagrams/ to dgm/, charts/ to chart/) and Microsoft_Excel_Worksheet.xlsx shortened to Excel_Worksheet.xlsx; parts outside word/ keep their full path, so _rels/.rels and customXml/item1.xml are unambiguous. **Editor names are shortened in the header only** — Fidus Writer to Fidus, canvas-editor + plugin to canvas + plugin, LibreOffice (baseline) to LibreOffice — while the prose and the summary table above keep the full names.

WordprocessingML and package parts

Package part	Plate.js	SuperDoc	Folio	GenOffice	eigenpal	office-open
[Content_Types].xml	C	P	P	P	P	w
_rels/.rels	C	P	P	P	w	w
customXml/item1.rels	D	P	P	P	w	P
customXml/item1.xml	D	P	P	P	w	P
customXml/itemProps1.xml	D	P	P	P	w	P
docProps/app.xml	D	P	P	P	w	C
docProps/core.xml	C	P	P	C	w	w
_rels/document.xml.rels	C	P	P	P	w	C
chart/_rels/chart1.xml.rels	D	P	P	P	w	w
comments.xml	D	w	P	w	C	C
document.xml	C	C	C	C	C	C
endnotes.xml	D	C	P	w	C	C
fontTable.xml	C	P	P	w	C	C
footer1.xml	D	C	P	w	C	C
footer2.xml	D	C	P	w	C	C
footnotes.xml	D	C	P	w	C	C
glossary/document.xml	D	P	P	w	C	C
header1.xml	D	C	P	w	C	C
header2.xml	D	C	P	w	C	C
numbering.xml	C	P	P	w	C	C
settings.xml	C	P	P	w	C	C

Package part	Plate.js	SuperDoc	Folio	GenOffice	eigenpal	office-open
styles.xml	C	P	P	w	C	C
theme/theme1.xml	C	P	P	P	w	w
webSettings.xml	C	P	P	w	C	C

WordprocessingML and package parts (continued)

Package part	Docen	WordCraft	Fidus	canvas + plugin	docs.plus	LibreOffice
[Content_Types].xml	C	C	C	C	C	C
_rels/.rels	w	P	C	C	C	C
customXml/item1.rels	P	P	D	D	D	C
customXml/item1.xml	P	P	D	D	D	w
customXml/itemProps1.xml	P	P	D	D	D	w
docProps/app.xml	C	C	C	C	D	C
docProps/core.xml	w	C	C	C	C	C
_rels/document.xml.rels	C	C	C	C	C	C
chart/_rels/chart1.xml.rels	w	P	D	D	D	D
comments.xml	C	C	D	C	D	C
document.xml	C	C	C	C	C	C
endnotes.xml	C	C	D	D	D	C
fontTable.xml	C	P	C	C	C	C
footer1.xml	C	C	D	C	D	C
footer2.xml	C	C	D	D	D	C
footnotes.xml	C	C	C	C	D	C
glossary/document.xml	C	P	D	D	D	w
header1.xml	C	C	D	C	D	C
header2.xml	C	C	D	D	D	C
numbering.xml	C	C	C	C	C	C
settings.xml	C	C	C	C	C	C
styles.xml	C	C	C	C	C	C
theme/theme1.xml	w	C	D	D	C	C
webSettings.xml	C	P	D	D	C	D

Binary payload and opaque parts

Package part	Plate.js	SuperDoc	Folio	GenOffice	eigenpal	office-open
chart/chart1.xml	D	P	P	P	w	w
chart/style1.xml	D	P	P	P	w	P
chart/colors1.xml	D	P	P	P	w	P
dgm/data1.xml	D	P	P	P	w	w
dgm/layout1.xml	D	P	P	P	w	w
dgm/quickStyle1.xml	D	P	P	P	w	w
dgm/colors1.xml	D	P	P	P	w	w
embed/embedded1.docx	D	P	P	P	P	P
embed/Excel_Worksheet.xlsx	D	P	P	P	P	P
media/image1.png	D	P	P	P	P	P
afchunk.html	D	P	P	P	P	P

Binary payload and opaque parts (continued)

Package part	Docen	WordCraft	Fidus	canvas + plugin	docs.plus	LibreOffice
chart/chart1.xml	w	P	D	D	D	C
chart/style1.xml	P	D	D	D	D	D
chart/colors1.xml	P	D	D	D	D	D
dgm/data1.xml	w	P	D	D	D	D
dgm/layout1.xml	w	P	D	D	D	D
dgm/quickStyle1.xml	w	P	D	D	D	D
dgm/colors1.xml	w	P	D	D	D	D
embed/embedded1.docx	P	P	D	D	D	D
embed/Excel_Worksheet.xlsx	P	P	D	D	D	P
media/image1.png	P	P	D	D	D	P
afchunk.html	P	D	D	D	D	D

The payload tables are the ones worth dwelling on. **word/afchunk.html is pass-through for seven editors and dropped by four.** The w:altChunk element that references it is dropped by three of the four that keep the part, which means the part survives as an orphan: the payload is in the package and nothing points at it. And **word/charts/style1.xml and word/charts/colors1.xml are dropped by WordCraft alone** among the editors that keep the chart — it copies the chart but not the chart's style and colour companions, which is why its output has a chart that renders differently from the original.

3. Three architectures

3.1 Clone-and-override — Folio

Folio's policy is stated in its own source (packages/core/src/docx/packageParts.ts:4-27):

> Folio models a handful of those parts — the body, the styles, the notes — and rewrites them on save. Everything else belongs to the author, and a repack hands it back byte for byte.

The mechanism is a clone of the source ZIP with only the modelled parts overridden (rezip.ts:1864, :1877, :1644), and the copy is **of the compressed bytes**: docx-core/src/zip/archive.ts:388-405 writes the raw deflate stream, so the CRC and the compressed size are preserved exactly. Parts that are never decoded stay that way (unzip.ts:951-960).

What Folio rebuilds: word/document.xml in full (rezip.ts:1999); every header and footer (:3141-3143); footnotes and endnotes, but by **splicing** rather than rebuilding (:3229-3236); styles.xml, by patching (:3722-3739); comments.xml, preserved untouched when unchanged (:473-477).

That is why Folio's part map has 34 pass-through parts: the corpus's headers, footers, notes, comments, chart, diagrams, embeddings, media, customXml, theme, fontTable and webSettings are all *the author's bytes, returned unchanged.*

3.2 Byte-splice of untouched blocks — GenOffice and eigenpal

Both keep the source document's XML as a sequence of blocks and re-serialize only the blocks they changed.

- **GenOffice** parses with packages/docx-engine/src/parse.ts:360 (parseDocx), converts to its editor model at apps/docs/src/renderer/editor/convert.ts:139 (blocksToPmDoc), plans the save at :2196 (pmDocToSavePlan), and applies it in packages/docx-engine/src/patch.ts:723 (saveDocx). The patcher splices the untouched blocks back **byte-exactly** (patch.ts:1647) and, for tables, falls back to copying the source part when its signature does not match the current model (patch.ts:2134). The result is re-zipped, not copied as compressed bytes.
- **eigenpal** reads the package with packages/core/src/store/package/ooxml-package.ts:418 (readOoxmlPackage) and writes it at :624 (writeOoxmlPackage). Its writer **starts from the original entry bytes and overwrites only the parts it holds as trees**, so binary and unmodelled XML parts are byte-preserved while the modelled ones are re-serialized in a normalized form.

The visible consequence is in the map: GenOffice has 13 parts that are *rewritten but semantically identical* — same tree, different bytes — and only 2 genuinely changed. eigenpal has 16.

3.3 Full regeneration — office-open, Docen, canvas-editor, docs.plus, Fidus Writer, WordCraft, Plate.js

These parse the document into a model and write a new package. Nothing survives by accident.

- **office-open** (packages/docx/src/parse.ts:501 → packages/docx/src/generate.ts:71) and **Docen** (which layers a Tiptap representation on top at packages/docx/src/converters/docx.ts:1337/:1415).
- **canvas-editor + plugin** rebuilds the package with the docx npm package (packages/docx/src/docx/exportDocx.ts:796), which brings its own default part set — styles, numbering, theme, settings, fontTable, webSettings, headers, footers — and writes the body from the canvas's element list.
- **docs.plus** runs DOCX → HTML (mammoth) → Tiptap JSON → HTML → DOCX (@turbodocx/html-to-docx).
- **Fidus Writer** does something categorically different: its exporter takes a **template URL** (src/exporter/docx/index.ts:34) and fills that template's placeholders (src/exporter/docx/render.ts:746-834, :982-1021; the placeholder vocabulary is defined in src/exporter/template/tags.ts:18-101 – {title}, {@<part-id>} for rich-text and table parts, plus {@bibliography}, {@copyright}, {@licenses}). The output package is the **template's**, not the input's. The exported word/styles.xml, word/numbering.xml and word/settings.xml are the template's, which is why they show as C while every input-specific part is D.
- **WordCraft**is a hybrid with a twist: crates/docx/src/package.rs:470-482 rebuilds the ZIP and **re-compresses everything** (deflate level 6; PNG, JPEG, GIF and WEBP are stored), so nothing is byte-identical at the ZIP level — yet 16 of its output parts are byte-identical *as content*, because binary payloads are copied through a manifest-gated passthrough (crates/docx/src/read/embed.rs:182 keep_embedded; crates/docx/src/write/embed.rs:297 copy_tree; crates/docx/src/write/embed.rs:186-189 available() requires both the manifest and the passthrough).

- **Plate.js** regenerates too, but through a different middle: importDocx runs **mammoth** into HTML and decodes that through the plugin set's own formats.html mappings (import/lib/importDocx.ts:2427), and exportDocx walks the model back out to **static HTML** and then to the DOCX writer (export/lib/exportDocx.tsx:646; export/lib/internal/html-to-docx.ts). So Plate's package is a projection of an **HTML document**, and the twenty-four parts HTML cannot carry simply have nowhere to go. Two things distinguish it from docs.plus, which has the same shape: it **diagnoses** every loss in a typed vocabulary rather than dropping it silently, and it keeps the original package alive under retainSource: true so that an unchanged document *may* be exported byte-exactly — a permission it grants only inside a **closed passive vocabulary** (internal/sourceEligibility.ts:453-582). On this corpus that permission is **refused**, with a reason:

  source-unavailable: The DOCX source cannot be retained because word/_rels/document.xml.rels declares an unrecognized relationship (chart); export will generate the document from editor content.

3.4 The opaque-blob fallback — SuperDoc

SuperDoc's engine is closed-source, but its contract is visible: SuperDoc.ts:4016-4023 contains a byte-identity fallback that returns the original blob untouched, and the engine's public type for parts it does not model is RawDocxPart with kind: 'added' | 'removed' | 'changed' and source: 'opaque-zip-entry' (docx-engine.d.ts:24-54). The engine also exposes three export modes — "review-preserving" | "final" | "original" (docx-engine.d.ts:73) — which is the correct shape for a round-trip tool, because "leave it exactly as it was" has to be an explicit choice.

---

4. Per editor: serialized, pass-through, rendered

4.1 SuperDoc — keeps everything, renders the least

Serialized. The engine holds the document as a parsed model and writes it back; parts it does not model survive as opaque ZIP entries.

Pass-through. 27 of 35 parts are byte-identical. The binary payload — chart, four SmartArt parts, the OLE embedding, the workbook, the image — is entirely byte-identical, as are customXml, webSettings, fontTable, theme and afchunk.html.

Rendered. Here is the finding that the 100% score conceals. SuperDoc classifies drawings into three support levels — supported, *fail-closed*, deferred (drawing-taxonomy.ts:33-42) — and the corpus exercises two of the fail-closed classes:

Construct	Support level	Consequence
Chart	supported	rendered as an inline SVG (chart-renderer.ts:1-11)
SmartArt / diagram	**fail-closed** (drawing-taxonomy.ts:327-333)	preserved in the file, **drawn as a placeholder**, diagnostic render.drawing.unsupported-object
Embedded OLE object	**fail-closed** (:320-326)	preserved in the file, **drawn as a placeholder**, diagnostic render.embedded-object-not-supported
VML shapes	fail-closed (:334-349)	preserved, placeholder

So SuperDoc is simultaneously the best round-tripper in the study and an editor that cannot show you two of the five things it round-trips perfectly. A user who opens a document with SmartArt in SuperDoc sees a labelled placeholder; a user who sends it on sees the real diagram, because the bytes were never touched.

4.2 Folio — copies the most, rewrites the least, and still gets one thing fatally wrong

Serialized. word/document.xml only, plus headers, footers, notes (spliced), styles (patched) and comments (unchanged).

Pass-through. 34 of 35 parts, as raw compressed bytes.

Rendered. Folio's model is ProseMirror. Two of its rendering decisions are visible in the file:

- mc:AlternateContent is captured verbatim and replayed on save **only while the node is untouched** (alternateContentAttrs.ts:28-69). That is a conditional guarantee, and it explains a measurable behaviour: the corpus's mc:Fallback text is in Folio's output.
- w14:paraId / w14:textId are lost. The mechanism is specific: the run parser applies the remainder attributes to w:r (runParser.ts:1509-1514), but parseRunProperties reads only the *children* of w:rPr (runParser.ts:696) and never applies a remainder to it. The attributes are therefore dropped, not deliberately removed.

Charts and SmartArt are rendered as **bounded previews**: the chart preview caps at 16 series and 64 points, the diagram preview at 128 shapes (chartPreview.ts, diagramPreview.ts). Constructs Folio cannot handle produce *named* diagnostics rather than silence — the warning catalogue is explicit (parseWarning.ts:22-67, parseWarningMessage.ts:36-80), including [Unsupported w:altChunk content] (altChunk.ts:10).

And one fatal defect. Folio's importer rewrites the move source's text as deleted text: the corpus's <w:moveFrom><w:r><w:t>moved away</w:t></w:r></w:moveFrom> comes out as <w:moveFrom><w:r><w:delText>moved away</w:delText></w:r></w:moveFrom>. w:delText means "this run contains literal text that shall be displayed when the run is part of a deletion" — a move source is not a deletion. **Microsoft Word 16.115 refuses the resulting file outright.** This is proved in the companion report, *Malformed Parts*; the proof is one attribute.

4.3 GenOffice — byte-exact splice, and two visible losses

Serialized. Only two parts genuinely change: word/document.xml and docProps/core.xml. Thirteen more are re-serialized to a semantically identical tree.

Pass-through. 20 parts, including the entire binary payload and afchunk.html.

Rendered. The editor's own model is what is rendered; the document generator is a separate concern from the renderer.

Its two measurable losses are both in the rendered result: the text inside two w:vMerge="continue" cells (98.8% text retention), and the fact that the mc:Fallback branch survives in the file but the mc:Choice branch is what the editor shows.

4.4 eigenpal — canonical tree, projection-only rendering

Serialized. A canonical typed model plus a generic OOXML tree. ooxml-package.ts:418 reads; :624 writes.

Pass-through. Only 5 parts are byte-identical — but the *content* of the binary payload is byte-identical, and the 16 "rewritten" parts are re-serialized to the same tree.

Rendered. eigenpal makes a distinction the other editors do not: **the MC branch selection is a projection of the render, not a mutation of the tree**. The file keeps both mc:Choice and mc:Fallback; the editor shows one. That is why eigenpal's output contains the fallback text even though the user never saw it — and it is why the first report's "flattened to mc:Choice" finding was an artifact of looking at the *rendered* result rather than the file.

w14:paraId is engine-owned and re-minted rather than preserved.

4.5 office-open and Docen — generators, with no renderer

office-open is a library. packages/docx/src/parse.ts:501 parses, packages/docx/src/generate.ts:71 generates. It has a renderer only in the sense that it can produce HTML for a browser preview; it does not draw a document.

Docen layers Tiptap on top (packages/docx/src/converters/docx.ts:1337, :1415). It keeps a coverage.ts registry that *documents* which OOXML features are preserved — including comments, endnotes, footnotes, bookmarks, headers, footers and pictures — but the registry's RUN_CHILDREN_PASSTHROUGH flag is never read. It is documentation, not enforcement.

Both drop opaque shapes to placeholders, both drop w16du:dateUtc and the w14 paragraph identifiers, and both produce the package-level defect analysed in the companion report.

4.6 WordCraft — regenerates the text, passes the payload through, and redraws the payload itself

WordCraft's module documentation states the design plainly (crates/docx/src/read/embed.rs:3-7):

> WordCraft draws charts and diagrams and shows an OLE object's picture, but doesn't edit them.

That is exactly the three-axis split:

Axis	WordCraft's behaviour	Evidence
Serialized	The body, styles, settings, numbering and all narrative parts are rebuilt	write/parts.rs:13
Pass-through	Binary payloads are copied through a manifest-gated passthrough; the ZIP itself is rebuilt and re-compressed	write/embed.rs:297, package.rs:470-482
Rendered	Charts and SmartArt are **redrawn from WordCraft's own model** as vector figures; an OLE object is shown as its stored picture	read/story/drawing.rs:223-234, crates/layout/src/display/lines.ts:592-596, read/story/vml.rs:340-341

So WordCraft is the one editor in this study that does **not** let the chart's own bytes decide what the user sees: the chart part is copied into the package byte-for-byte, but the canvas draws a chart **reconstructed from WordCraft's model**. The keep-vs-regenerate fork at read/story/drawing.rs:223-234 is the decision point.

That split has a cost the part map records precisely: WordCraft keeps word/charts/chart1.xml but **drops word/charts/style1.xml and word/charts/colors1.xml**. The chart survives; the chart's style and colour companions do not. It is the only editor that keeps the chart and loses its companions, and that is why its chart renders differently from the original while the part still measures as pass-through.

Two of its losses are forced by that choice: mc:AlternateContent is collapsed to a single branch, with the mc:Fallback discarded (read/story/runs.rs:100-107), and w:fldSimple is converted into a complex field (read/story/runs.rs:42-49). And w14:paraId is written only for comment paragraphs (write/story/para.rs:55-58).

4.7 Fidus Writer — the output is the template's

Fidus is not a round-tripper in any sense, and its architecture explains why. The exporter takes a **template URL** (src/exporter/docx/index.ts:34); the template is a separate uploaded file (fiduswriter/style/models.py:69-70); the exporter fills the template's placeholders (src/exporter/docx/render.ts:746-834, :982-1021). The result is written through JSZip with everything re-deflated (src/exporter/tools/xml_zip.ts:60-66, :125-136).

Its importer is equally blunt, and it is honest about it:

Corpus construct	Fidus behaviour	Evidence
Tables	**dropped entirely** — the block converter accepts only w:p	src/importer/docx/convert.ts:739-742
Charts, SmartArt, OLE	**dropped** — the figure converter requires an a:blip	src/importer/docx/convert.ts:925-933
Content controls	only citation SDTs are handled	src/importer/docx/convert.ts:1196
Everything else unrecognized	logged, not rendered	console.warn("unhandled node", child) at :1211-1213

The importer logs twelve "unhandled element" classes. That logging is a real virtue: Fidus is the only editor in this study that tells you, at the point of loss, what it did not understand.

4.8 canvas-editor ✦ plugin — a bespoke parser and a third-party writer

canvas-editor writes neither the importer nor the exporter of a typical editor: its DOCX importer is a **bespoke 1,859-line OOXML parser over JSZip** (packages/docx/src/docx/importDocx.ts:1807) and its exporter **rebuilds the package with the `docx` npm package** (packages/docx/src/docx/exportDocx.ts:796).

There is no copy path at all. exportDocx.ts:1-54 imports from docx, color and ./utils — never jszip; jszip appears only in the importer (importDocx.ts:15, :1810). The exporter never opens the source package. The body is rebuilt from the editor's **in-memory element list** (exportDocx.ts:809), and the package is produced entirely by Packer.toBlob (:917).

That makes the output the docx package's default part set (docx@8.6.0, build/index.mjs:19 031-19 381): [Content_Types].xml, _rels/.rels, word/document.xml and its rels, styles.xml, numbering.xml, settings.xml, fontTable.xml, footnotes.xml, comments.xml, docProps/core.xml, docProps/app.xml, docProps/custom.xml, and — **no word/theme/theme1.xml and no word/webSettings.xml**. Everything specific to the corpus is gone.

Two details worth noting because they are easy to misread from a part map:

- **Media is re-emitted under a random name.** The docx package names each image ${nanoid()}.${type} (build/index.mjs:11 191, :10 332), so the corpus's word/media/image1.png reappears as word/media/zsakc18onfp21xzadj5t0.png — byte-identical content under a random new name. A part map that only compares *names* would score this as a drop plus an add; the bytes are the same.
- **A surviving part is not a surviving feature.** word/footnotes.xml survives but holds only Word's two boilerplate notes; word/comments.xml survives with **zero** w:comment elements; word/styles.xml is regenerated from docx's DefaultStylesFactory (:15 543-15 553), so the corpus's TableGrid style is gone and the exporter writes explicit borders instead (exportDocx.ts:524).

And the canvas renders by **crashing on vertical merges**. The importer does read w:vMerge, but drops continuation cells outright (importDocx.ts:1738), which shifts every later cell in the row one column left; the renderer then indexes past the end of the column group at TableParticle.ts:724:

```
TypeError: Cannot read properties of undefined (reading 'width')
  at Object.computeRowColInfo (…/TableParticle.ts:724:45)
  at LayoutEngine.computeRowList (…/LayoutEngine.ts:296:33)
  at Draw.render (…/Draw.ts:2664:44)
  at Command.executeImportDocx (…/importDocx.ts:1857:13)
```

The exception is caught by the harness, which then exports a document that is missing the table content. The trace is reproducible and is stored in out/canvas/import_error.txt.

The canvas also **loses the inserted text of tracked changes** — not just the revision markup. parseParagraph handles only w:r, w:sdt, w:fldSimple, w:hyperlink and w:fldChar; the w:ins wrapper is skipped, so its text goes with it. And nothing is reported: there is no console.warn anywhere in packages/docx/src/.

4.9 docs.plus — the document becomes HTML and comes back

docs.plus's pipeline is DOCX → HTML → Tiptap JSON → HTML → DOCX.

mammoth is the importer (domain/docxImport.ts:43, :70-73), and the exporter is @turbodocx/html-to-docx (domain/docxExport.ts:30, :39). The source package is out of scope at export time: http/controller.ts:84-93 passes only the Tiptap document, a title and media coordinates, and docxExport.ts:39 passes null for the header HTML — so the converter is never told the original document existed. It writes its own default part set and emits **no headers and footers at all**.

What mammoth does to tracked changes is worth stating precisely, because it is not what the first report said. mammoth/lib/docx/body-reader.js has a reader table that takes priority over its ignoreElements map:

Construct	Fate
w:ins (:447), w:moveTo (:450)	**read** — the text survives as ordinary visible HTML; the revision markup does not
w:del (:729), w:moveFrom (:732)	in ignoreElements — **silently dropped**, no warning
deleted paragraph mark (:287-290)	the paragraph's children are merged into the next paragraph, as Word does it
deleted table row (:499-503)	the row is dropped from the table

So mammoth **flattens tracked changes to their 'accepted' state**: insertions and move-to text remain visible as ordinary text, deletions and move-from text are removed, and the revision *markup* — author, date, ids, the elements themselves — is destroyed. ignoreElements is not a skip list but a **warning-suppression** list, consulted at exactly one place (body-reader.js:51-52): an element in the map has no reader and vanishes silently; an element *not* in the map without a reader vanishes with the warning "An unrecognised element was ignored: " + element.name, which docs.plus maps to its own unsupported-element diagnostic (domain/docxImport.ts:33-36).

Other corpus features: charts and SmartArt are **silently dropped** (the drawing reader matches only pic:pic→a:blip, body-reader.js:463-464, :592-600); the OLE object is dropped **with** a warning; **comments are dropped silently, body and marker both**, because convertCommentReference is wrapped in an ignore path that never runs (lib/document-to-html.js:297-298 + lib/styles/html-paths.js:71-75); footnotes and endnotes survive as body text with dead anchor links rather than as real notes; mc:AlternateContent is rendered from its mc:Fallback (:393-395).

The consequence shows up in the rendered comparison: docs.plus produces **92 rendered lines where the corpus has 81**, with 61 lines that were never in the corpus. HTML has no page or section model, so the three sections, the page borders and the landscape orientation all collapse, and the tables reflow into HTML tables that render differently.

4.10 Plate.js — an HTML-shaped editor with a byte-exactness gate

Serialized. Everything. importDocx(arrayBuffer, { plugins, lossPolicy, retainSource }) (import/lib/importDocx.ts:2427) parses the package with @zip.js/zip.js, runs **mammoth** into HTML, and decodes that HTML through the plugin set's formats.html mappings into Plate's editor model. The plugin set is the serialiser's *schema*: the harness runs **45 base plugins** — paragraph, heading, the eight mark plugins, the six style plugins, table/row/cell, link, image, media/audio/video/file, media-embed, list, code-block, indent, mention, toc, footnote, comments, callout, column group/item, details, date, upload, and the two equation plugins.

Pass-through. None. **Zero parts are byte-identical**; 24 of 35 are dropped and the remaining 11 are regenerated. The reason is structural and it is worth stating precisely: the DOCX pipeline is **DOCX → HTML → model → HTML → DOCX**, and the HTML step cannot carry a header, footer, footnote, endnote, comment, glossary part, customXml part, altChunk part, chart, SmartArt diagram, OLE object, embedded workbook or an image reference. Every one of those leaves the package.

Plate has one more mechanism that no other editor here has, and it is the most interesting thing in this report's §4: **retainSource: true keeps the admitted source package alive**, and internal/sourceEligibility.ts:453-582 decides whether the export may reuse it byte-for-byte. The gate is a **closed passive vocabulary**: exactly one officeDocument relationship pointing at word/document.xml (:531-542), no external relationships except absolute hyperlinks (:500-514), raster parts matching their declared image type's magic bytes (:543-547), every part reachable through the relationship graph (:560-567), and only PAGE recognised as a field instruction (:128). The corpus **fails** that gate — word/_rels/document.xml.rels declares a chart relationship, which is not passive — so Plate refuses the byte-exact path and regenerates, saying exactly why:

> source-unavailable: The DOCX source cannot be retained because word/_rels/document.xml.rels declares an unrecognized relationship (chart); export will generate the document from editor content.

This is the only implementation of *provable* byte-exact export in the whole study, and it is also the only one that **declines to use it** on this document rather than quietly copying the bytes.

Rendered. The weakest result in the trial, with one exception. The output renders in **2 pages** against the corpus's 6, with **39 missing** and **12 extra** rendered lines — it loses the page model, the sections, the headers and footers, and the chart, SmartArt and OLE entirely, and it loses the **image** as well (Plate's import reports resource-omitted: An embedded image was omitted during DOCX import, twice).

What survives is the text, and the text survives well: **91.7%**, the second-highest in the trial. The table grid, gridSpan, vMerge, nested tables and cell shading all survive; bookmarks survive; w:vertAlign survives.

The one lever that changes the result. exportDocx takes a presentation option (export/lib/exportDocx.tsx:646; export/lib/internal/docxStaticComponents.tsx:200-211), which maps ten plugins to DOCX-native drawings — callout to a two-cell table, columns to a layout table, equations to KaTeX→MathML→OMML, TOC to hyperlink paragraphs. When it is omitted, the export falls back to each plugin's own React component, and the **Base** (non-React) plugins carry none, so tables, images and lists silently degrade to plain divs (static/internal/staticPresentation.ts:103). The harness therefore uses Plate's own static kit from apps/www/src/registry/components/editor/*-static.tsx (32 files) with DOCX_EXPORT_STYLES, exactly as Plate's own demo does. **A DOCX harness that omits presentation is measuring Plate's fallback, not Plate.**

4.11 The second axis that matters: what the editor *says*

Plate is the only editor in the study with a **typed diagnostic vocabulary**, and that changes what a scorecard can mean. Its import emits **38 diagnostics** — 19 unsupported-content, 6 converter-message, 2 resource-omitted, 8 property-dropped, 2 lossy-content, 1 source-unavailable — each naming a specific construct:

Diagnostic	What Plate said about the corpus
unsupported-content ×19	field-instruction content replaced; field content replaced ×2; five structured-document-tags unwrapped; embedded-object content replaced; text-box content unwrapped; alt-chunk content dropped; header/footer/footnote/endnote content not mapped into the main document; four move-range markers flattened
converter-message ×6	w:cr, w:fldSimple, o:OLEObject, w:permStart, w:permEnd, w:altChunk unrecognised
resource-omitted ×2	an embedded image was omitted during DOCX import
property-dropped ×8	background color ×2, font size ×4, font family ×2, caps, small caps, character spacing, alignment, superscript
lossy-content ×2	tracked revisions were flattened into the proposed document; comment 0 has no complete imported range
source-unavailable ×1	the source cannot be retained because the .rels declares an unrecognized relationship (chart)

And lossPolicy **defaults to 'reject'** (import/lib/importDocx.ts:2455), so by default those diagnostics are 18 **errors** and the import **fails**: Plate refuses the corpus rather than writing a degraded copy of it. The harness runs with 'allow' only so that an output exists to measure.

No other editor in this trial reports anything. Fidus, canvas-editor and docs.plus drop twenty-odd parts in silence; SuperDoc, Folio, GenOffice and eigenpal preserve them but say nothing about the constructs they cannot model; office-open and Docen preserve them faithfully and still cannot open their own output in Word.

---

5. The cross-cutting finding: kept but not rendered

Editor	Chart	SmartArt	OLE object	Image	Rendered faithfully?
SuperDoc	rendered as SVG	**placeholder** (fail-closed)	**placeholder** (fail-closed)	rendered	partly
Folio	bounded preview (16 series / 64 points)	bounded preview (128 shapes)	preview	rendered	partly
GenOffice	preserved	preserved	preserved	rendered	no SmartArt/OLE rendering found in the generator path
eigenpal	preserved	preserved, branch projected at render	preserved	rendered	partly
office-open	preserved, **but its relationship**	preserved, relationship dangles	preserved	rendered	no

Editor	Chart	SmartArt	OLE object	Image	Rendered faithfully?
	dangles				
Docen	placeholder	placeholder	placeholder	rendered	no
WordCraft	**redrawn from its own model**	**redrawn from its own model**	shown as its stored picture	rendered	yes, for charts
Fidus Writer	dropped	dropped	dropped	rendered	no
canvas-editor	dropped	dropped	dropped	rendered	no
docs.plus	dropped	dropped	dropped	rendered	no
Plate.js	dropped	dropped	dropped	**dropped**	no

Plate.js is the only editor in the trial that drops the image. Its import reports resource-omitted: An embedded image was omitted during DOCX import — twice, for the two a:blip references in the corpus — and the image part does not survive into the output. docs.plus, canvas-editor and Fidus all keep the image; Plate, whose pipeline is HTML-shaped, does not.

This table is the reason the three axes have to be measured separately. **The editors that preserve the most are frequently not the ones that show the most**, and the fidelity score of the first report is silent about the difference because it measures the file, not the screen.

LibreOffice is not in this table. Every other row is a file:line claim against a pinned commit in repos/; LibreOffice is not one of the fifteen projects in the study, so there is no source evidence to cite for its render path, and inventing one by inference would be exactly the kind of claim this report refuses to make. Its behaviour is measured two ways that *are* reproducible — as the renderer in §6 and as the round-trip baseline in the first report — and that is where its numbers live.

---

6. The rendered comparison

build/render_compare.py renders the corpus and every output to PDF, extracts the laid-out text, and diffs against the corpus's own rendering.

Corpus: **81 lines, 265 words, 6 pages.**

Output	Lines	Words	Pages	Missing	Extra
SuperDoc	81	265	6	**0**	**0**
Folio	81	265	6	**0**	**0**
eigenpal	81	265	6	**0**	**0**
LibreOffice (baseline)	81	265	6	0	0
GenOffice	81	262	6	1	1
WordCraft	79	260	6	5	3
office-open	69	251	6	**12**	4
Docen	67	244	6	**14**	5
canvas-editor + plugin	45	179	**2**	41	14
Plate.js	**39**	**168**	**2**	**39**	12
Fidus Writer	30	139	**2**	49	13
docs.plus	92	263	**3**	43	**61**

(The corpus renders in 6 pages. The pages column is the clearest single indicator of what a pipeline did to the document: 6 means the page model survived, 3 means the sections did not.)

Two rows need a caveat before the rest can be read.

The LibreOffice row is a control, not a result. LibreOffice is the renderer that produced every PDF in this table, so its own output rendering back to itself is trivially perfect. The 0/0 row says the *measurement* is sound — the renderer and the diff are not losing lines — and it must not be read as a claim about LibreOffice's fidelity, which the companion report measures separately and finds to be 86.0%.

Four of these outputs are files Microsoft Word will not open. Folio, office-open, Docen and Fidus Writer are refused by Word 16.115, for reasons set out in the companion report. Their rows are still measured, because a rendering is a rendering whether or not Word agrees with it — but a row of "0 missing" for Folio is not a recommendation to ship a Folio output.

Two readings matter.

office-open loses 12 rendered lines while scoring 100% text retention on the XML. The text is all still in word/document.xml; the chart is still there as an element; but the relationship that binds the chart to the package dangles, so nothing draws it. The first report's checker cannot see this, because a checker reads XML and a reader looks at a page. The companion report proves the cause.

docs.plus renders 11 lines more than the corpus contains. Nothing was invented — the corpus's 81 lines are still there in a different order — but HTML's model has no sections, no page borders and no landscape, so the document is reflowed into 3 pages and the tables become HTML tables.

---

7. What this means

- **'Supported' is a claim about a file, a screen and a pipeline, and they are different claims.** SuperDoc supports SmartArt in the file and not on screen. Folio supports tables in the file and renders them. WordCraft keeps the chart's bytes in the package but drops the chart's style and colour companions, and draws a chart of its own instead of the original one.
- **Pass-through is the strongest fidelity and the weakest editing.** An editor that copies 34 parts byte-for-byte cannot round-trip a change to any of them, and the parts it does rebuild are the only ones it can change.
- **The parts an editor does not model are the parts a user will not see.** Fidus Writer's exporter writes the *template's* styles, numbering and settings; canvas-editor's exporter writes the docx package's defaults; docs.plus's pipeline has no page model at all. In each case the output is a valid document of the right shape that is not the document the user opened.
- **Choose on the axis you care about.** If you need the document to survive a round trip, Folio and SuperDoc. If you need to *see* charts and SmartArt, WordCraft, which redraws them from its own model, and not SuperDoc, which preserves them and shows a placeholder. If you need a document Microsoft Word will open, note that **four of the twelve outputs in this study are refused by Word 16.115** — Folio, office-open, Docen and Fidus Writer — for reasons set out in the companion report. Two of those four pass every static check.

Reproducing this

```
build/part_map.py              # package-level serialization map
build/part_map.py --matrix     # the full part x editor matrix
build/part_map.py --detail     # per-part listing for one editor
build/part_matrix_md.py        # the matrix as four readable tables
build/split_report_matrix.py   # splices those tables into this report
build/render_compare.py        # rendered-text diff of every output
build/doc_diff.py <editor-dir>  # element-level diff of document.xml
build/payload_audit.py         # payload parts and body references per editor
build/make_report.sh <report.md>  # markdown -> .docx, with the two Word fixes
```

build/make_report.sh is the pipeline that produces this file: pandoc writes w:dirty="true" on the table-of-contents field, which Word 16.115 refuses, so build/fix_dirty.py rewrites it; and pandoc sizes every table for a portrait page, so build/retune_tables.py re-fits the tables to the landscape content width.

The per-editor code citations in §4 refer to the pinned commits listed in the first report, under repos/. None of those checkouts was modified.